

Tarea 5.1.- Creación de chatbots con Python y spaCy.

¿Qué es un chatbot?

Un chatbot es un programa de software que simula una conversación humana mediante texto o voz. Su propósito es asistir al usuario en tareas específicas o simplemente interactuar de forma automática.

Tipos de chatbots:

1. Basados en reglas: Utilizan flujos de decisión predefinidos (if/else).
2. Basados en aprendizaje automático (ML): Aprenden a partir de datos y pueden entender lenguaje natural con más flexibilidad.
3. Basados en NLP (Procesamiento del Lenguaje Natural): Usan técnicas como reconocimiento de entidades, análisis de intención, etc.

Componentes esenciales de un chatbot con NLP:

- Tokenización: Separar texto en palabras o frases.
- Lematización/Stemming: Reducir palabras a su forma base.
- Reconocimiento de intención (intent recognition): Detectar qué quiere hacer el usuario.
- Reconocimiento de entidades (NER): Identificar nombres, fechas, lugares, etc.
- Manejo de diálogo: Responder de forma coherente.

Actividad guiada. Chatbot con Python y spaCy.

Objetivo: Crear un chatbot basado en reglas, pero con capacidades NLP para detectar la intención del usuario.

Realizar en un Google Colab los pasos indicados a continuación.

Paso 1: Instalación de librerías necesarias

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

Paso 2: Importar librerías y cargar modelo NLP

```
import spacy

# Cargar modelo de lenguaje inglés
nlp = spacy.load("en_core_web_sm")
```

Paso 3: Definir intenciones y respuestas

```
intents = {
    "greeting": ["hello", "hi", "hey", "good morning", "good afternoon"],
    "goodbye": ["bye", "goodbye", "see you", "farewell"],
    "thanks": ["thanks", "thank you", "appreciate it"],
    "name": ["what is your name", "who are you"]
}

responses = {
    "greeting": "Hello! How can I assist you today?",
    "goodbye": "Goodbye! Have a great day!",
    "thanks": "You're welcome!",
    "name": "I'm a simple chatbot built with Python and spaCy."
}
```

Paso 4: Procesar el texto de entrada con NLP

```
def get_intent(user_input):
    text = user_input.lower().strip()
    for intent, keywords in intents.items():
        for keyword in keywords:
            if keyword in text:
                return intent
    return "unknown"
```

Paso 5: Generar respuesta del chatbot

```
def get_response(user_input):
    intent = get_intent(user_input)
    return responses.get(intent, "Sorry, I didn't understand that.")
```

Paso 6: Simular el chatbot en consola

```
print("Chatbot is running! (Type 'exit' to stop)")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        print("Chatbot: Bye!")
        break
    response = get_response(user_input)
    print("Chatbot:", response)
```

Ampliaciones posibles

- Uso de similitud semántica con ``doc.similarity``.
- Uso de ``Matcher`` de spaCy para patrones más complejos.
- Entrenamiento de modelos personalizados de intención.
- Integración con interfaces gráficas (Tkinter, Flask) o servicios de mensajería (Telegram, Discord).